

claude-windows / 42211f80-2169-4025-ad9d-99c1aa29ff70

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\42211f80-2169-4025-ad9d-99c1aa29ff70\subagents\workflows\wf_d4359b2a-456\agent-ac8cf20670daa232b.jsonl`
- SHA-256: `772cee00553058b52cd1236a03e5235323e3d45ef130ff450eb2f347d2730b6a`
- Source modified: `2026-06-06T15:22:47+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_d4359b2a-456`
- session_id: `42211f80-2169-4025-ad9d-99c1aa29ff70`
```

Transcript

1. user (2026-06-06T15:20:37.523Z)

You are mapping ONE subsystem of the "badminton-highlight-indexer" Python repo (cwd is the repo root) for a dense CODE MAP that a future AI engineer will read to ramp up FAST on the codebase without crawling files.

Subsystem: Sports, providers, annotations, DB
Files: backend/sports/base.py, backend/sports/badminton.py, backend/sports/__init__.py, backend/providers/base.py, backend/providers/gemini.py, backend/providers/__init__.py, backend/annotations/base.py, backend/annotations/file_provider.py, backend/annotations/automated_provider.py, backend/annotations/__init__.py, backend/database.py

READ each listed file (use Read; use Grep/Glob to resolve any directory globs to actual files). Then return, per file:

- path (exact, repo-relative, forward slashes)
- role: one terse line ? what the file is for
- symbols: the IMPORTANT classes / functions / dataclasses / module-level constants & registries. For each: name, kind (class|func|dataclass|const|registry), a terse signature (params + return where useful), and a one-line role. SKIP trivial private helpers and boilerplate.
- contracts: exact data shapes this file defines or consumes (dict keys, CSV column orders, tuple layouts, dataclass fields, return types). Be precise ? these are the cross-module interfaces.
- edges: calls/imports to OTHER files that matter for dataflow, each as "thisSymbol -> otherFile.thing (why)".
- gotchas: non-obvious constraints, footguns, platform/WSL/GPU/threadpool specifics, dead paths, defaults that bite.

Rules: be terse and ACCURATE. Every symbol and path MUST actually exist in the file ? do NOT invent or guess. If something is notable but you're unsure, omit it. Return via the structured schema.

2. assistant (2026-06-06T15:20:40.454Z)

I'll map this subsystem. Let me read all the listed files in parallel.

3. user (2026-06-06T15:20:41.019Z)

```
1     from abc import ABC, abstractmethod
2
3     class SportProfile(ABC):
4         @property
5         @abstractmethod
6         def name(self) -> str:
7             """The identifier of the sport (e.g., 'badminton')."""
8             pass
```

```

9
10     @abstractmethod
11     def get_prompt(self, chunk_duration: float = 0.0) -> str:
12         """Full system prompt for the AI model, including output schema."""
13         pass
14
15     @abstractmethod
16     def get_relevance_config(self) -> dict:
17         """Sport-specific visual validation params (court colors, thresholds, etc.)."""
18         pass
19

```

4. user (2026-06-06T15:20:41.898Z)

```

1     from backend.sports.base import SportProfile
2
3     class BadmintonSportProfile(SportProfile):
4         @property
5         def name(self) -> str:
6             return "badminton"
7
8         def get_prompt(self, chunk_duration: float = 0.0) -> str:
9             base_prompt = """System Role & Objective
10             You are a highly precise computer vision data extraction engine specializing in
11             badminton match analysis. Your exclusive objective is to parse a full-length match video and
12             generate a flawless JSON array containing exact rally boundaries and shot metrics for automated
13             highlight generation.
14
15             Core Directives
16             Detect every single active rally in the video. Do not miss any live gameplay.
17             Extract high-precision timestamps as TOTAL DECIMAL SECONDS from the start of the video
18             for exact frame-accurate cuts.
19             Estimate the number of shots exchanged (net crossings) and provide a confidence score
20             for that count, acknowledging frame-sampling limitations.
21
22             CRITICAL TIMESTAMP FORMAT RULES:
23             - All start_time and end_time values MUST be expressed as TOTAL SECONDS from the
24             beginning of the video, as a decimal number.
25             - Example: A rally starting at 1 minute 14 seconds into the video MUST be written as
26             74.0, NOT as "1:14" or 114.
27             - Example: A rally starting at 2 minutes 30.5 seconds MUST be written as 150.5, NOT as
28             "2:30.5" or 230.5.
29             - NEVER use MM:SS or HH:MM:SS colon-separated format. NEVER concatenate minutes and
30             seconds into a single number without converting.
31             - The downstream pipeline will BREAK if timestamps are not in total decimal seconds.
32
33             Positive Constraints: Rally Boundaries
34             Start Condition: The exact fractional second the server's racket makes contact with the
35             shuttlecock.
36             End Condition: The rally concludes ONLY when both of the following physical criteria are
37             met sequentially: First, the shuttlecock hits the court surface, strikes the net and drops, or
38             lands visibly out of bounds. Second, a 0.3 to 0.5-second buffer has elapsed where both players
39             have visibly ceased all kinetic momentum (no lunging, sliding, or racket follow-through).
40
41             Negative Constraints: Anomaly Rejection (CRITICAL)
42             Replay Rejection: You must strictly ignore and filter out all slow-motion instant
43             replays. Replays are not live rallies.
44             Dead-Time Rejection: Ignore all Hawkeye/line-call challenges, interval coaching, towel
45             breaks, court mopping, changing shuttles, and player celebrations.
46             Pre-serve Rituals: Do not start the timestamp during the players' setup or stance. Wait
47             for physical racket-to-shuttle contact.
48
49             Shot Tracking & Data Confidence
50             Shot Definition: Count every successful traverse of the shuttlecock across the net
51             (serves, clears, smashes, drops, drives). Do not count the final grounding or pre-net faults.

```

```

35     Confidence Scoring: Because the shuttlecock moves at extreme velocities that may fall
between sampled frames, provide a shot_count_confidence metric (Low, Medium, High). Base your
estimate on player biomechanics, positioning, and racket swings if the shuttle itself is
blurred.
36
37     Strict Output Schema
38     Output strictly in the following JSON array format. Do not include markdown formatting,
introductory text, conversational filler, or explanations. Any deviation from this schema will
break the downstream pipeline.
39     Remember: start_time and end_time are TOTAL SECONDS (decimal), not MM:SS.
40
41     [
42     {
43     "start_time": 74.0,
44     "end_time": 81.3,
45     "shuttle_exchange_count": 11,
46     "shot_count_confidence": "High"
47     },
48     {
49     "start_time": 150.5,
50     "end_time": 174.2,
51     "shuttle_exchange_count": 24,
52     "shot_count_confidence": "Medium"
53     }
54     ]
55     """
56     if chunk_duration > 0.0:
57         duration_context = (
58             f"IMPORTANT VIDEO CONTEXT: This video segment is exactly
{chunk_duration:.1f} seconds long. "
59             f"All timestamps MUST be between 0 and {chunk_duration:.1f}. "
60             f"Any timestamp beyond {chunk_duration:.1f}s is INVALID.\n\n"
61         )
62         return duration_context + base_prompt
63         return base_prompt
64
65     def get_relevance_config(self) -> dict:
66         return {
67             "court_green_lower": [35, 40, 40],
68             "court_green_upper": [85, 255, 255],
69             "court_pixels_min_ratio": 0.05
70         }
71

```

5. user (2026-06-06T15:20:42.602Z)

```

1     from typing import Dict, Type
2     from backend.sports.base import SportProfile
3     from backend.sports.badminton import BadmintonSportProfile
4
5     class SportRegistry:
6         def __init__(self):
7             self._registry: Dict[str, Type[SportProfile]] = {}
8             # Automatically register Badminton
9             self.register(BadmintonSportProfile)
10
11         def register(self, profile_cls: Type[SportProfile]):
12             inst = profile_cls()
13             self._registry[inst.name] = profile_cls
14
15         def get(self, name: str) -> SportProfile:
16             if name not in self._registry:
17                 raise KeyError(f"Sport '{name}' is not registered in the SportRegistry.
Available: {list(self._registry.keys())}")
18             return self._registry[name]()

```

```

19
20     def list_available(self) -> Dict[str, str]:
21         return {name: cls.__doc__ or f"{name.capitalize()} sport profile" for name, cls
in self._registry.items()}
22
23     sport_registry = SportRegistry()
24

```

6. user (2026-06-06T15:20:43.369Z)

```

1     from abc import ABC, abstractmethod
2     from typing import Tuple
3
4     class AIVideoProvider(ABC):
5         def __init__(self, api_key: str, model_name: str):
6             self.api_key = api_key
7             self.model_name = model_name
8
9         @abstractmethod
10        def analyze_video(self, video_path: str, prompt: str,
11                          chunk_duration: float = 0.0,
12                          label: str = "") -> Tuple[list, dict]:
13            """Sends a video file to the AI model along with a prompt, and returns a tuple
of
14            (segments_list, metrics_dict).
15
16            Args:
17                video_path: Path to the video file to analyze.
18                prompt: Prompt instructions.
19                chunk_duration: Duration of this chunk in seconds, or 0.0 if not chunked.
20                label: Diagnostic label for logging.
21
22            Returns:
23                Tuple containing:
24                - A list of detected segment dicts (e.g. [{"start_time": ..., "end_time":
...}])
25                - A dictionary of performance metrics (e.g. {"upload_time": ...,
"process_time": ..., "gen_time": ...})
26            """
27            pass
28

```

7. user (2026-06-06T15:20:43.638Z)

```

1     import time
2     import json
3     import logging
4     from typing import Tuple
5     from google import genai
6     from google.genai import types
7     from backend.providers.base import AIVideoProvider
8
9     logger = logging.getLogger(__name__)
10
11    class GeminiProvider(AIVideoProvider):
12        def __init__(self, api_key: str, model_name: str):
13            super().__init__(api_key, model_name)
14            # Initialize client lazily to avoid connection/auth issues during class
instantiation
15            self._client = None
16
17        @property
18        def client(self):
19            if self._client is None:
20                self._client = genai.Client(api_key=self.api_key)

```

```

21         return self._client
22
23     def analyze_video(self, video_path: str, prompt: str,
24                       chunk_duration: float = 0.0,
25                       label: str = "") -> Tuple[list, dict]:
26         log_prefix = f"Gemini {label}" if label else "Gemini"
27
28         metrics = {
29             "upload_time": 0.0,
30             "process_time": 0.0,
31             "gen_time": 0.0
32         }
33
34         # 1. Upload
35         logger.info(f"[{log_prefix}] Uploading {video_path}...")
36         t_upload_start = time.time()
37         try:
38             video_file = self.client.files.upload(file=video_path)
39             metrics["upload_time"] = time.time() - t_upload_start
40             logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
41         except Exception as e:
42             logger.error(f"[{log_prefix}] Upload failed: {e}")
43             return [], metrics
44
45         # 2. Wait for processing on Google servers
46         t_process_start = time.time()
47         try:
48             while video_file.state.name == "PROCESSING":
49                 logger.info(f"[{log_prefix}] Processing on Google servers. Elapsed:
{time.time()-t_process_start:.1f}s. Retrying in 10s...")
50                 time.sleep(10)
51                 video_file = self.client.files.get(name=video_file.name)
52                 if video_file.state.name == "FAILED":
53                     logger.error(f"[{log_prefix}] Video processing failed:
{video_file.error.message}")
54                     try:
55                         self.client.files.delete(name=video_file.name)
56                     except Exception:
57                         pass
58                     return [], metrics
59         except Exception as e:
60             logger.error(f"[{log_prefix}] Status check failed: {e}")
61             try:
62                 self.client.files.delete(name=video_file.name)
63             except Exception:
64                 pass
65             return [], metrics
66
67         metrics["process_time"] = time.time() - t_process_start
68         logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")
69
70         # 3. Call Gemini
71         logger.info(f"[{log_prefix}] Submitting multimodal request to
{self.model_name}...")
72         t_gen_start = time.time()
73         raw_text = None
74         try:
75             response = self.client.models.generate_content(
76                 model=self.model_name,
77                 contents=[video_file, prompt],
78                 config=types.GenerateContentConfig(
79                     response_mime_type="application/json",
80                     temperature=0.0

```

```

81         )
82     )
83     raw_text = response.text
84     metrics["gen_time"] = time.time() - t_gen_start
85     logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")
86     except Exception as e:
87         logger.error(f"[{log_prefix}] API call failed: {e}")
88     finally:
89         try:
90             self.client.files.delete(name=video_file.name)
91         except Exception:
92             pass
93
94     if not raw_text:
95         return [], metrics
96
97     # 4. Parse JSON
98     try:
99         segments = json.loads(raw_text)
100         if not isinstance(segments, list):
101             logger.error(f"[{log_prefix}] Response is not a JSON array.")
102             return [], metrics
103         return segments, metrics
104     except Exception as e:
105         logger.error(f"[{log_prefix}] JSON parse failed: {e}")
106         return [], metrics
107

```

8. user (2026-06-06T15:20:44.780Z)

```

1     from typing import Dict, Type
2     from backend.providers.base import AIVideoProvider
3     from backend.providers.gemini import GeminiProvider
4
5     class ProviderRegistry:
6         def __init__(self):
7             self._registry: Dict[str, Type[AIVideoProvider]] = {}
8             # Register Gemini provider
9             self.register("gemini", GeminiProvider)
10
11         def register(self, name: str, provider_cls: Type[AIVideoProvider]):
12             self._registry[name] = provider_cls
13
14         def get(self, name: str, api_key: str, model_name: str) -> AIVideoProvider:
15             if name not in self._registry:
16                 raise KeyError(f"AI Provider '{name}' is not registered in the
ProviderRegistry. Available: {list(self._registry.keys())}")
17             return self._registry[name](api_key=api_key, model_name=model_name)
18
19         def list_available(self) -> Dict[str, str]:
20             return {name: cls.__doc__ or f"{name.capitalize()} provider" for name, cls in
self._registry.items()}
21
22     provider_registry = ProviderRegistry()
23

```

9. user (2026-06-06T15:20:44.910Z)

```

1     """Annotation provider registry ? pluggable sources of ground-truth rally labels.
2
3     An `AnnotationProvider` turns a video reference into a list of `(start, end)` rally
4     intervals ? the same shape that `backend.eval.annotations.load_annotations`
5     produces and `backend.eval.metrics` consumes. That symmetry is the whole point:
6     golden-set ground truth can come from a CSV file, a human annotation vendor, or an

```

```

7     automated oracle behind one interface, so the regression flow
8     (`backend.eval.regression`) never needs to know where labels came from.
9
10    Mirrors the existing registry pattern in `backend/providers/base.py` and
11    `backend/indexer/base.py`. See `docs/GOLDEN_SET_VENDORING.md` (and ADR-006) for the
12    design rationale.
13    """
14    from abc import ABC, abstractmethod
15    from typing import Dict, List, Optional, Tuple, Type
16
17    # An (start, end) interval in seconds ? identical to backend.eval.metrics.Interval.
18    Interval = Tuple[float, float]
19
20    # Job lifecycle statuses returned by poll().
21    QUEUED = "queued"
22    RUNNING = "running"
23    DONE = "done"
24    FAILED = "failed"
25    STATUSES = (QUEUED, RUNNING, DONE, FAILED)
26
27
28    class AnnotationProvider(ABC):
29        """Abstract source of ground-truth rally intervals for a video.
30
31        The three-method contract (submit ? poll ? fetch) accommodates both
32        synchronous providers (a file already on disk) and asynchronous ones (a human
33        vendor that takes days). Synchronous providers can use the `annotate()`
34        convenience, which runs the full cycle in one call.
35        """
36
37        @property
38        @abstractmethod
39        def name(self) -> str:
40            """Unique registry key for this provider (e.g. 'file', 'vendor', 'auto')."""
41
42        @abstractmethod
43        def submit(self, video_ref: str, guideline: Optional[str] = None,
44                  sport: str = "badminton") -> str:
45            """Submit a video for annotation; return an opaque job_id.
46
47            `video_ref` is provider-defined (a file path, a video_id/MD5, a URL).
48            `guideline` is the annotation spec (what counts as a rally start/end).
49            """
50
51        @abstractmethod
52        def poll(self, job_id: str) -> str:
53            """Return one of STATUSES for the given job."""
54
55        @abstractmethod
56        def fetch(self, job_id: str) -> List[Interval]:
57            """Return the completed annotations as sorted `(start, end)` intervals.
58
59            Should raise if the job is not DONE / the labels are unavailable, rather
60            than returning an empty list (which would silently skew downstream scores).
61            """
62
63        def annotate(self, video_ref: str, guideline: Optional[str] = None,
64                  sport: str = "badminton") -> List[Interval]:
65            """Convenience: submit + fetch for a provider that completes synchronously.
66
67            Async providers (e.g. a human vendor with a multi-day SLA) should be driven
68            via submit/poll/fetch explicitly instead ? calling this on one will raise
69            once the job isn't immediately DONE.
70            """
71            job_id = self.submit(video_ref, guideline=guideline, sport=sport)

```

```

72         status = self.poll(job_id)
73         if status != DONE:
74             raise RuntimeError(
75                 f"{self.name}.annotate(): job '{job_id}' is '{status}', not DONE. "
76                 f"Use submit/poll/fetch for asynchronous providers."
77             )
78         return self.fetch(job_id)
79
80
81     class AnnotationProviderRegistry:
82         """Registry of annotation providers, keyed by each provider's `name`."""
83
84         def __init__(self):
85             self._providers: Dict[str, Type[AnnotationProvider]] = {}
86
87         def register(self, provider_cls: Type[AnnotationProvider]) ->
Type[AnnotationProvider]:
88             """Register a provider class under its `name`. Returns the class (decorator-
friendly)."""
89             try:
90                 key = provider_cls().name
91             except Exception:
92                 # Providers needing constructor args fall back to the class name.
93                 key = getattr(provider_cls, "__name__", "unknown")
94             self._providers[key] = provider_cls
95             return provider_cls
96
97         def get(self, name: str, **kwargs) -> AnnotationProvider:
98             """Instantiate the named provider, passing through any constructor kwargs."""
99             cls = self._providers.get(name)
100             if cls is None:
101                 raise KeyError(
102                     f"Unknown annotation provider '{name}'. Available:
{self.list_available()}"
103                 )
104             return cls(**kwargs)
105
106         def list_available(self) -> List[str]:
107             return list(self._providers.keys())
108
109
110     annotation_registry = AnnotationProviderRegistry()
111

```

10. user (2026-06-06T15:20:45.454Z)

```

1     """FileProvider ? ground-truth rally labels from pre-made CSV files.
2
3     This formalizes today's manual / ShuttleSet annotation flow behind the
4     `AnnotationProvider` interface: the "annotation job" is simply a CSV that already
5     exists on disk (the same `start,end` format `run_eval.py --scaffold` writes). The
6     job_id IS the resolved CSV path, so submit/poll/fetch all operate on that path.
7
8     It's the synchronous, zero-cost provider used as the default golden-set source and
9     as the stand-in in tests/CI before a human-vendor or automated provider is wired in.
10    """
11    import os
12    from typing import List, Optional
13
14    from backend.annotations.base import (
15        AnnotationProvider,
16        annotation_registry,
17        Interval,
18        DONE,
19        FAILED,

```



```

20 )
21 from backend.eval.annotations import load_annotations
22
23
24 class FileProvider(AnnotationProvider):
25     def __init__(self, csv_dir: Optional[str] = None, ext: str = ".csv"):
26         """
27         Args:
28             csv_dir: if set, a bare `video_ref` (not itself a path) is resolved to
29                     `

```

11. user (2026-06-06T15:20:45.818Z)

```

1  """AutomatedProvider ? the fast-tier "oracle" annotation source (GS-4, interface stub).
2
3  The fast/CI tier of the golden-set flow needs an *automated* labeler that turns a
4  video into rally `(start, end)` intervals quickly and cheaply, as a smoke alarm
5  between authoritative human-vendor runs (the slow tier).
6
7  This slice ships the **interface only**, deliberately model-agnostic: an
8  `AutomatedProvider` that delegates labeling to an injected `labeler` callable, plus
9  a deterministic fake for tests/CI. No concrete model is wired yet ? that's a
10 tracked backlog item (docs/BACKLOG.md), to be chosen alongside the production LLM
11 step so the lineages differ (see the oracle rule below).
12
13 ?? **Oracle rule.** The labeler MUST be a *different model lineage* than whatever
14 production uses. If production becomes the offline ActionFormer segmenter and the

```

```

15     oracle is also ActionFormer, they share blind spots and the regression test is
16     blind ? green while both are wrong. `lineage` records the oracle's model family so
17     callers can guard against it matching production (`warn_if_same_lineage`).
18     """
19     import logging
20     from typing import Callable, Dict, List, Optional
21
22     from backend.annotations.base import (
23         AnnotationProvider,
24         annotation_registry,
25         Interval,
26         DONE,
27         FAILED,
28     )
29
30     logger = logging.getLogger(__name__)
31
32     # A labeler turns (video_ref, guideline, sport) into rally intervals.
33     Labeler = Callable[[str, Optional[str], str], List[Interval]]
34
35
36     def _not_configured(video_ref: str, guideline: Optional[str], sport: str) ->
List[Interval]:
37         """Default labeler: refuses to run until a real model is injected.
38
39         Keeps `AutomatedProvider()` constructible (so the registry can key it as 'auto')
40         while making accidental use of an unconfigured oracle fail loudly rather than
41         silently returning no rallies.
42         """
43         raise NotImplementedError(
44             "AutomatedProvider has no labeler configured. Inject one via "
45             "annotation_registry.get('auto', labeler=<fn>, lineage='<model-family>'). "
46             "Choosing the concrete oracle model is a tracked item ? see docs/BACKLOG.md
(GS-4)."
47         )
48
49
50     def warn_if_same_lineage(oracle_lineage: Optional[str], production_lineage:
Optional[str]) -> bool:
51         """Warn (and return True) if the oracle shares production's model lineage.
52
53         Encodes the oracle rule in code, not just docs: a same-lineage oracle can't be
54         trusted to catch production's failures. Callers wiring CI should pass both.
55         """
56         if oracle_lineage and production_lineage and oracle_lineage == production_lineage:
57             logger.warning(
58                 "Oracle lineage '%s' matches production's ? the regression test may be "
59                 "blind to shared failure modes. Use a different model lineage.",
60                 oracle_lineage,
61             )
62             return True
63         return False
64
65
66     class AutomatedProvider(AnnotationProvider):
67         def __init__(self, labeler: Optional[Labeler] = None, lineage: str = "unspecified"):
68             """
69             Args:
70                 labeler: callable producing rally intervals for a video. Defaults to a
71                         stub that raises until configured.
72                 lineage: the oracle's model family (e.g. 'gemini', 'qwen-vl') ? recorded
73                         so callers can enforce the oracle rule via warn_if_same_lineage.
74             """
75             self._labeler = labeler or _not_configured
76             self.lineage = lineage

```

```

77         # In-process synchronous job store: job_id -> labeled intervals.
78         self._jobs: Dict[str, List[Interval]] = {}
79
80     @property
81     def name(self) -> str:
82         return "auto"
83
84     def submit(self, video_ref: str, guideline: Optional[str] = None,
85                sport: str = "badminton") -> str:
86         # Synchronous stub: run the labeler now and stash the result. A real async
87         # oracle (e.g. a queued API job) would kick off here and resolve in poll().
88         intervals = self._labeler(video_ref, guideline, sport)
89         intervals = sorted((float(s), float(e)) for s, e in intervals)
90         job_id = video_ref
91         self._jobs[job_id] = intervals
92         return job_id
93
94     def poll(self, job_id: str) -> str:
95         return DONE if job_id in self._jobs else FAILED
96
97     def fetch(self, job_id: str) -> List[Interval]:
98         if job_id not in self._jobs:
99             raise KeyError(f"AutomatedProvider: unknown job '{job_id}' (submit it
first)")
100         return self._jobs[job_id]
101
102
103     annotation_registry.register(AutomatedProvider)
104

```

12. user (2026-06-06T15:20:46.768Z)

```

1     """Pluggable sources of ground-truth rally annotations.
2
3     `AnnotationProvider`s yield `(start, end)` intervals ? the same shape the eval
4     harness scores ? so golden-set labels can come from a file, a human vendor, or an
5     automated oracle behind one interface. See `docs/GOLDEN_SET_VENDORING.md`.
6     """
7     from .base import annotation_registry, AnnotationProvider
8     from . import file_provider      # noqa: F401 (side-effect: registers FileProvider)
9     from . import automated_provider # noqa: F401 (side-effect: registers
AutomatedProvider)
10
11     __all__ = ["annotation_registry", "AnnotationProvider"]
12

```

13. user (2026-06-06T15:20:47.057Z)

```

1     import sqlite3
2     import json
3     import os
4     import logging
5     from typing import List, Dict, Any, Optional
6
7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         def _get_connection(self):
15             conn = sqlite3.connect(self.db_path)
16             conn.row_factory = sqlite3.Row
17             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.

```

```

18         conn.execute("PRAGMA foreign_keys = ON")
19         return conn
20
21     def _init_db(self):
22         """Initializes tables for videos, play_segments, and events."""
23         with self._get_connection() as conn:
24             cursor = conn.cursor()
25
26             # Videos Table
27             cursor.execute("""
28                 CREATE TABLE IF NOT EXISTS videos (
29                     id TEXT PRIMARY KEY,
30                     filepath TEXT NOT NULL,
31                     processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
32                     status TEXT NOT NULL,
33                     validation_results TEXT
34                 )
35             """)
36
37             # Play Segments Table (Extensible metadata JSON)
38             cursor.execute("""
39                 CREATE TABLE IF NOT EXISTS play_segments (
40                     id INTEGER PRIMARY KEY AUTOINCREMENT,
41                     video_id TEXT NOT NULL,
42                     start_time REAL NOT NULL,
43                     end_time REAL NOT NULL,
44                     duration REAL NOT NULL,
45                     server TEXT,
46                     receiver TEXT,
47                     metadata TEXT,
48                     FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
49                 )
50             """)
51
52             # Events Table
53             cursor.execute("""
54                 CREATE TABLE IF NOT EXISTS events (
55                     id INTEGER PRIMARY KEY AUTOINCREMENT,
56                     segment_id INTEGER NOT NULL,
57                     timestamp REAL NOT NULL,
58                     type TEXT NOT NULL,
59                     player TEXT,
60                     details TEXT,
61                     FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
62                     CASCADE
63                 )
64             """)
65
66             # Versioned migrations live here. PRAGMA user_version is the schema
67             # contract ? bump it for every change and add an ordered `if version < N`
68             # block below. Tests assert the expected version, so accidental drift fails
69
70             version = cursor.execute("PRAGMA user_version").fetchone()[0]
71
72             if version < 1:
73                 # v1: raw candidate detections (pre-confirmation), detector-agnostic.
74                 # Common fields are columns; detector-specific metrics go in `stats`
75                 # so a new segmenter reuses this table by setting its own
76                 # `detector`/`stats`.
77                 cursor.execute("""
78                     CREATE TABLE IF NOT EXISTS candidate_segments (
79                         id INTEGER PRIMARY KEY AUTOINCREMENT,
80                         video_id TEXT NOT NULL,
81                         detector TEXT NOT NULL,

```

```

79             chunk_index INTEGER,
80             start_time REAL NOT NULL,
81             end_time REAL NOT NULL,
82             duration REAL NOT NULL,
83             confidence REAL,
84             stats TEXT,
85             detection_params TEXT,
86             confirmed_segment_id INTEGER,
87             human_label TEXT,
88             notes TEXT,
89             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
90             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
91             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
92         )
93     """)
94     cursor.execute("""
95         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
96         ON candidate_segments(video_id, detector)
97     """)
98     cursor.execute("PRAGMA user_version = 1")
99     # future: if version < 2: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 2
100
101     conn.commit()
102
103     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
104         try:
105             with self._get_connection() as conn:
106                 conn.cursor().execute(
107                     "INSERT OR REPLACE INTO videos (id, filepath, status,
validation_results) VALUES (?, ?, ?, ?)",
108                     (video_id, filepath, status, json.dumps(validation_results))
109                 )
110                 conn.commit()
111                 return True
112         except Exception as e:
113             logger.error(f"Failed to add video: {e}")
114             return False
115
116     def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
117         try:
118             with self._get_connection() as conn:
119                 cursor = conn.cursor()
120                 cursor.execute(
121                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
122                     (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
123                 )
124                 conn.commit()
125                 return cursor.lastrowid
126         except Exception as e:
127             logger.error(f"Failed to add play segment: {e}")
128             return None
129
130     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
131         try:
132             with self._get_connection() as conn:
133                 conn.cursor().execute(
134                     "INSERT INTO events (segment_id, timestamp, type, player, details)

```

```

VALUES (?, ?, ?, ?, ?)",
135             (segment_id, timestamp, event_type, player, json.dumps(details or
136             {})))
137             )
138             conn.commit()
139             return True
140         except Exception as e:
141             logger.error(f"Failed to add event: {e}")
142             return False
143
144     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
145                               end_time: float,
146                               chunk_index: Optional[int] = None, confidence:
147                               Optional[float] = None,
148                               stats: Optional[Dict[str, Any]] = None,
149                               detection_params: Optional[Dict[str, Any]] = None) ->
150                               Optional[int]:
151         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
152         detector-specific dicts stored as JSON. Returns the new candidate id, or
153         None."""
154         try:
155             with self._get_connection() as conn:
156                 cursor = conn.cursor()
157                 cursor.execute(
158                     """INSERT INTO candidate_segments
159                     (video_id, detector, chunk_index, start_time, end_time, duration,
160                     confidence, stats, detection_params)
161                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
162                     (video_id, detector, chunk_index, start_time, end_time, end_time -
163                     start_time,
164                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
165                     {})))
166                 )
167                 conn.commit()
168                 return cursor.lastrowid
169         except Exception as e:
170             logger.error(f"Failed to add candidate segment: {e}")
171             return None
172
173     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
174         """Record that a candidate detection was confirmed as a given play_segment."""
175         try:
176             with self._get_connection() as conn:
177                 conn.cursor().execute(
178                     "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
179                     ?",
180                     (segment_id, candidate_id)
181                 )
182                 conn.commit()
183                 return True
184         except Exception as e:
185             logger.error(f"Failed to link candidate {candidate_id} to segment
186             {segment_id}: {e}")
187             return False
188
189     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
190                                  only_unlabeled: bool = False) -> List[Dict[str, Any]]:
191         """Fetch candidate detections for the annotation / fine-tuning workflow.
192         `stats` and `detection_params` are deserialized from JSON."""
193         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
194         params: List[Any] = [video_id]
195
196         if detector:
197             query += " AND detector = ?"
198             params.append(detector)

```

```

190         if only_unlabeled:
191             query += " AND human_label IS NULL"
192
193         query += " ORDER BY start_time"
194
195         candidates = []
196         with self._get_connection() as conn:
197             rows = conn.cursor().execute(query, params).fetchall()
198             for row in rows:
199                 d = dict(row)
200                 d["stats"] = json.loads(d["stats"] or "{}")
201                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
202                 candidates.append(d)
203         return candidates
204
205     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
206         with self._get_connection() as conn:
207             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
208 (video_id,)).fetchone()
209             if row:
210                 d = dict(row)
211                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
212                 return d
213             return None
214
215     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
216 List[Dict[str, Any]]:
217         """
218         Dynamically queries play segments based on filters:
219         - duration_min: float
220         - server: string
221         - receiver: string
222         - event_type: string (e.g. smash, foul)
223         """
224         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
225         params = [video_id]
226
227         if filters.get("duration_min"):
228             query += " AND r.duration >= ?"
229             params.append(filters["duration_min"])
230
231         if filters.get("server"):
232             query += " AND r.server = ?"
233             params.append(filters["server"])
234
235         if filters.get("receiver"):
236             query += " AND r.receiver = ?"
237             params.append(filters["receiver"])
238
239         if filters.get("event_type"):
240             # Check if there is an event of a specific type inside this segment
241             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
242 e.type = ?)"
243             params.append(filters["event_type"])
244
245         segments_list = []
246         with self._get_connection() as conn:
247             cursor = conn.cursor()
248             rows = cursor.execute(query, params).fetchall()
249
250             for row in rows:
251                 r_dict = dict(row)
252                 r_id = r_dict["id"]
253                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")

```

```

252             # Fetch associated events
253             event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
254             r_dict["events"] = []
255             for er in event_rows:
256                 e_dict = dict(er)
257                 e_dict["details"] = json.loads(e_dict["details"] or "{}")
258                 r_dict["events"].append(e_dict)
259
260             segments_list.append(r_dict)
261
262             return segments_list
263
264         def clear_video_data(self, video_id: str):
265             with self._get_connection() as conn:
266                 conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
267                 conn.commit()
268

```

14. assistant (2026-06-06T15:20:50.297Z)

I have all the files. Let me check `backend/eval/annotations.load\_annotations` to confirm the consumed contract.

15. user (2026-06-06T15:20:51.167Z)

```

1         """Load hand-annotated ground-truth rally timestamps from CSV.
2
3         Annotation file format (one file per video):
4
5             start,end
6             00:01:12,00:01:39
7             102.5,131.0
8
9         - A header row `start,end` is optional (auto-detected and skipped).
10        - Each timestamp is either decimal seconds (`102.5`) or colon-separated
11        `MM:SS` / `HH:MM:SS` (`00:01:12`). Both forms may be mixed in one file.
12        - Blank lines and `#` comment lines are ignored.
13        """
14
15        import csv
16        import logging
17        from typing import List, Tuple
18
19        logger = logging.getLogger(__name__)
20
21        Interval = Tuple[float, float]
22
23
24        def parse_timestamp(value: str) -> float:
25            """Parse a timestamp string to float seconds.
26
27            Accepts decimal seconds or colon-separated MM:SS / HH:MM:SS.
28            Raises ValueError on anything unparseable so bad annotations fail loudly.
29            """
30            value = value.strip()
31            if not value:
32                raise ValueError("empty timestamp")
33            if ":" in value:
34                parts = value.split(":")
35                if len(parts) > 3:
36                    raise ValueError(f"too many ':' segments in timestamp '{value}'")
37                total = 0.0
38                for part in parts:                                # most-significant first (H, M, S)
39                    total = total * 60 + float(part)

```



```

40         return total
41     return float(value)
42
43
44 def load_annotations(csv_path: str) -> List[Interval]:
45     """Load and validate ground-truth rally intervals from a CSV file.
46
47     Returns intervals sorted by start time. Raises ValueError on malformed rows
48     (bad timestamp, start >= end) so labeling mistakes surface immediately rather
49     than silently skewing scores.
50     """
51     intervals: List[Interval] = []
52     with open(csv_path, newline="") as f:
53         reader = csv.reader(f)
54         for lineno, row in enumerate(reader, start=1):
55             if not row:
56                 continue
57             first = row[0].strip()
58             if not first or first.startswith("#"):
59                 continue
60             # Skip an optional header row.
61             if first.lower() in ("start", "start_time"):
62                 continue
63             if len(row) < 2:
64                 raise ValueError(f"{csv_path}:{lineno}: expected 'start,end', got
{row!r}")
65             try:
66                 start = parse_timestamp(row[0])
67                 end = parse_timestamp(row[1])
68             except ValueError as e:
69                 raise ValueError(f"{csv_path}:{lineno}: {e}") from e
70             if start >= end:
71                 raise ValueError(f"{csv_path}:{lineno}: start ({start}) >= end ({end})")
72             intervals.append((start, end))
73
74     intervals.sort(key=lambda iv: iv[0])
75     logger.info("Loaded %d ground-truth rallies from %s", len(intervals), csv_path)
76     return intervals
77
78
79 def write_scaffold(csv_path: str) -> None:
80     """Write an empty annotation file with just the header, for hand-filling."""
81     with open(csv_path, "w", newline="") as f:
82         writer = csv.writer(f)
83         writer.writerow(["start", "end"])
84

```

16. assistant (2026-06-06T15:20:53.899Z)

I have everything needed. Returning the structured map.

17. user (2026-06-06T15:22:41.076Z)

Structured output provided successfully

18. assistant (2026-06-06T15:22:47.707Z)

Mapped the subsystem. All 11 files read (plus `backend/eval/annotations.py` to confirm the consumed CSV contract), and the structured map returned via StructuredOutput.

Key cross-cutting takeaways for a future engineer:

- Three identical registry-singleton patterns (`sport\_registry`, `provider\_registry`, `annotation\_registry`), but they differ in how keys are assigned: sports/annotations derive the key from an instance `.name`; providers pass the key string explicitly to `register()`.

- The `(start, end)` `Interval` tuple in `backend/annotations/base.py` is intentionally identical to `backend.eval.annotations`/`backend.eval.metrics` ? that shape parity is the whole point of the annotation provider abstraction.
- `backend/annotations/\_\_init\_\_.py` import side-effects are load-bearing (they register `file` and `auto`); the noqa imports cannot be removed.
- `database.py` re-issues `PRAGMA foreign\_keys = ON` per connection (SQLite default is off), and schema changes must bump `PRAGMA user\_version` or CI fails.